Chartly

Data Analysis Platform

Final Project Report

MVP Version 2.0

August 2026

Prepared by: Chartly Development Team

Repository: github.com/ArwaZ2002/Chartly

Live Deployment: chartly-sigma.vercel.app

Table of Contents

1. Executive Summary 3
2. Project Overview 3
3. Features 4
4. Technology Stack 5
5. Architecture 6
6. Database Schema 7
7. API Reference 8
8. Security 10
9. Frontend 11
10. Testing 12
11. Deployment 13
12. Known Limitations 14
13. Future Improvements 14

1. Executive Summary

Chartly is a web-based data analysis platform that enables users to upload CSV and Excel files, explore automatic data profiles, and generate interactive charts powered by Plotly. The platform supports role-based access control (Admin, Analyst, Viewer), chart versioning, templates, audit trails, and code export in both Python (Plotly Express) and JavaScript (Plotly.js).

The Minimum Viable Product (MVP) delivers a fully functional single-page application with a FastAPI backend, SQLAlchemy ORM, SQLite (local) or PostgreSQL (production) database, and a Vanilla JavaScript frontend. It is deployed on Vercel's free tier with Neon Postgres as the production database, requiring zero infrastructure cost.

Key metrics: 34 API endpoints, 9 database tables, 10 chart types, 25 automated tests, and a comprehensive security layer including JWT authentication, Argon2id password hashing, rate limiting, CSP headers, and CSRF protection.

2. Project Overview

2.1 Purpose

Chartly was designed as an internal data analysis platform to bridge the gap between raw data and actionable visual insights. Users can upload datasets, receive automatic profiling (column types, statistics, quality flags), and build charts through an interactive builder without writing code.

2.2 Goals

- Zero-cost deployment using Vercel free tier + Neon Postgres
- No external dependencies for charting (Plotly.js CDN)
- Role-based access: Admin manages users, Analyst creates charts, Viewer reads only
- Full audit trail for compliance and collaboration
- Code export for reproducibility (Python and JavaScript)
- Responsive design for desktop and mobile

2.3 Target Users

Business analysts, data scientists, and product managers who need quick visual exploration of CSV/Excel data without setting up local Python environments or BI tools.

3. Features

3.1 Authentication & Authorization

- JWT-based authentication with HttpOnly cookies
- Argon2id password hashing (auto-migration from legacy SHA-256)
- Three roles: Admin, Analyst, Viewer with endpoint-level enforcement
- Token revocation (logout invalidates both cookie and bearer tokens)
- Rate limiting: 5 failed logins per account, 20 per IP per 60s window

3.2 Dataset Management

- Upload CSV, XLSX, and XLS files (up to 50 MB)
- Automatic profiling: column types (numeric, categorical, datetime, boolean), statistics, missing values
- Preview first 10 rows with type annotations
- In-database file storage (Postgres LargeBinary) for serverless compatibility
- Rename and delete datasets with cascade to charts

3.3 Chart Builder

- 10 chart types: Bar, Line, Area, Pie, Scatter, Histogram, Box, Violin, Treemap, Funnel
- Interactive builder: dimension, metric, aggregation (sum, mean, count, min, max)
- Smart recommendations based on column types
- Chart regeneration with specification changes
- Version history with restore capability
- Rendered as interactive Plotly.js visualizations

3.4 Templates & Themes

- Save chart configurations as reusable templates
- Apply templates to new datasets
- Custom theme templates with color palettes and font selection
- Light/dark theme toggle with localStorage persistence

3.5 Code Export

- Python export: Plotly Express code ready to run
- JavaScript export: Plotly.js code for web integration
- Exported code includes data, configuration, and layout

3.6 Admin Features

- User management (CRUD) with role assignment
- Audit trail: tracks chart creation, regeneration, export, and template saves
- Platform settings (name, default chart type)
- Security: API docs auto-disabled in production, HSTS, CSP, CORS

4. Technology Stack

4.1 Backend

Component	Technology	Version
Web Framework	FastAPI	0.141.1
ASGI Server	Uvicorn	0.52.1
ORM	SQLAlchemy	2.0.52
Validation	Pydantic + pydantic-settings	2.13.4
Auth	PyJWT + argon2-cffi	2.10.1 / 25.1.0
Data Processing	pandas + openpyxl + xlrd	3.0.5
Charting	Plotly (Python)	6.9.0
Database (prod)	psycopg (PostgreSQL)	3.3.4
Database (dev)	SQLite3 (built-in)	--

4.2 Frontend

Component	Technology
Framework	Vanilla JavaScript (ES6 Modules, SPA)
Charting	Plotly.js 2.35.0 (CDN)
Routing	Custom hash-based router (#login, #dashboard, etc.)
Styling	Custom CSS (design system with CSS variables)
Fonts	Inter + Poppins (Google Fonts)
State	localStorage (theme), JWT cookie (auth)

4.3 Infrastructure

Service	Provider	Tier
Hosting	Vercel	Free (Hobby)
Database	Neon Postgres	Free (0.5 GB)
Version Control	GitHub	Free
CDN	Vercel Edge Network	Included
CI/CD	Vercel Git Integration	Automatic

5. Architecture

5.1 System Overview

Chartly follows a monolithic architecture with clear separation of concerns. The FastAPI application handles both API routes and static file serving. On Vercel, static assets are served from the CDN (public/ directory) while the Python function handles API requests.

5.2 Request Flow

1. Browser requests / -> Vercel CDN serves public/index.html
2. SPA loads, JS fetches /api/auth/login -> Vercel routes to FastAPI function
3. FastAPI processes request, returns JWT in HttpOnly cookie
4. Subsequent API calls include cookie -> FastAPI validates JWT
5. Chart generation: FastAPI calls data_io to read from DB, pandas for profiling, Plotly for chart HTML
6. Response returned to SPA, Plotly.js renders interactive chart

5.3 Directory Structure

```
chartly-mvp/
  api/index.py          Vercel serverless entrypoint
  backend/
    main.py              FastAPI app, middleware, seeding
    app/
      api/               Route handlers (8 modules)
      core/              Config, database, rate limiter
      models/            SQLAlchemy + Pydantic models
      services/          Business logic (chart gen, profiling)
    tests/               Test suite (25 tests)
    static/              SPA assets (local dev)
  public/                Vercel CDN-served assets
    index.html           SPA shell
    static/css/          Stylesheets
    static/js/           JavaScript modules
  vercel.json            Vercel configuration
  requirements.txt       Python dependencies
```

5.4 Key Design Decisions

- File storage in DB (LargeBinary): Avoids ephemeral disk issues on serverless platforms
- Inline HTML in main.py: Ensures index.html is in the function bundle for Vercel
- Hash-based SPA routing: No server-side routing needed, works with CDN-only static files
- Import-time DB init: Runs on Vercel cold start, idempotent seeding

6. Database Schema

Chartly uses 9 database tables with UUID primary keys. The schema supports user management, dataset storage, chart configurations, versioning, templates, and audit logging.

users

Column	Type	Constraints
id	String	PK, UUID
email	String	Unique, indexed
name	String	Not null
hashed_password	String	Argon2id
role	String	admin/analyst/viewer
created_at	DateTime	Default utcnow

datasets

Column	Type	Constraints
id	String	PK, UUID
name	String	Not null
file_name	String	Original filename
file_data	LargeBinary	Raw file bytes
columns_profile	JSON	Auto-profiled schema
row_count	Integer	Total rows
uploaded_by	String	FK -> users.id

charts

Column	Type	Constraints
id	String	PK, UUID
dataset_id	String	FK -> datasets.id
specification	JSON	Chart config
created_by	String	FK -> users.id

chart_versions

Column	Type	Constraints
id	String	PK, UUID
chart_id	String	FK -> charts.id
specification	JSON	Historical spec

templates

Column	Type	Constraints
id	String	PK, UUID

name

String

Template name

chart_type

String

bar/line/etc

specification

JSON

Saved config

theme_templates

Column	Type	Constraints
id	String	PK, UUID
name	String	Theme name
colors	JSON	Color palette
font_family	String	Optional font

audit_entries

Column	Type	Constraints
id	String	PK, UUID
chart_id	String	FK -> charts.id
action	String	create/export/etc
user_id	String	FK -> users.id

settings

Column	Type	Constraints
key	String	PK
value	Text	Setting value

revoked_tokens

Column	Type	Constraints
jti	String	PK, JWT ID
expires_at	DateTime	TTL index

7. API Reference

Chartly exposes 34 endpoints across 8 route groups. All endpoints except login/health require JWT authentication. Role-based access is enforced at the route level.

Group	Method	Path	Auth	Description
Auth	POST	/api/auth/login	No	Login, returns JWT cookie
Auth	POST	/api/auth/logout	Optional	Revoke token, clear cookie
Auth	GET	/api/auth/me	Any	Current user profile
Datasets	POST	/api/datasets/upload	Admin, Analyst	Upload CSV/XLSX
Datasets	GET	/api/datasets/	Any	List all datasets
Datasets	GET	/api/datasets/{id}	Any	Get dataset details
Datasets	PATCH	/api/datasets/{id}	Admin, Analyst	Rename dataset
Datasets	GET	/api/datasets/{id}/preview	Any	Preview first 10 rows
Datasets	DELETE	/api/datasets/{id}	Admin, Analyst	Delete dataset
Charts	POST	/api/charts/generate	Admin, Analyst	Generate chart
Charts	GET	/api/charts/	Any	List charts
Charts	GET	/api/charts/recommendations/{id}	Any	Chart suggestions
Charts	GET	/api/charts/{id}/render	Any	Render chart HTML+JSON
Charts	POST	/api/charts/{id}/regenerate	Admin, Analyst	Update spec
Charts	POST	/api/charts/{id}/restore/{vid}	Admin, Analyst	Restore version
Charts	DELETE	/api/charts/{id}	Admin, Analyst	Delete chart
Templates	POST	/api/templates/	Admin, Analyst	Create template
Templates	GET	/api/templates/	Any	List templates
Templates	GET	/api/templates/{id}	Any	Get template
Templates	POST	/api/templates/apply	Admin, Analyst	Apply template
Templates	DELETE	/api/templates/{id}	Admin, Analyst	Delete template
Themes	POST	/api/theme-templates/	Admin, Analyst	Create theme
Themes	GET	/api/theme-templates/	Any	List themes
Themes	DELETE	/api/theme-templates/{id}	Admin, Analyst	Delete theme
Audit	GET	/api/audit/	Admin	List audit entries
Users	GET	/api/users/	Admin	List users
Users	POST	/api/users/	Admin	Create user
Users	PATCH	/api/users/{id}	Admin	Update user
Users	DELETE	/api/users/{id}	Admin	Delete user
Settings	GET	/api/settings/	Any	Get settings
Settings	PUT	/api/settings/	Admin	Update settings
Settings	GET	/api/settings/theme	Any	Get theme preference
Settings	PUT	/api/settings/theme	Any	Update theme preference

8. Security

8.1 Authentication

- JWT tokens stored in HttpOnly, SameSite=Lax cookies (not accessible via JavaScript)
- Bearer token support for API clients (Authorization header)
- Token expiration: 120 minutes, configurable via ACCESS_TOKEN_EXPIRE_MINUTES
- Token revocation: logout invalidates JWT ID (jti) in revoked_tokens table

8.2 Password Hashing

- Argon2id (argon2-cffi) for new passwords - memory-hard, GPU-resistant
- Automatic migration from legacy SHA-256 hashes on successful login
- Legacy hashes preserved after failed login to prevent user lockout

8.3 Rate Limiting

- Login endpoint: 5 failures per account per 60s window (configurable)
- IP-based limit: 20 failures per IP per 60s window
- Retry-After header returned on 429 responses

8.4 Security Headers

- X-Content-Type-Options: nosniff
- X-Frame-Options: DENY (prevents clickjacking)
- Referrer-Policy: strict-origin-when-cross-origin
- Content-Security-Policy: restricts script/style/connect sources
- Strict-Transport-Security: enabled when COOKIE_SECURE=true (HTTPS)

8.5 CSRF Protection

- Origin header validation on POST/PUT/PATCH/DELETE requests
- Rejects requests from unknown origins (allows same-origin and API clients)

8.6 Input Validation

- File type validation: only .csv, .xlsx, .xls allowed
- MIME type checking on upload
- Max upload size: 50 MB (configurable)
- Pydantic validation on all API inputs
- SQLAlchemy ORM prevents SQL injection

9. Frontend

9.1 Architecture

The frontend is a single-page application (SPA) built with Vanilla JavaScript using ES6 modules. It uses a custom hash-based router for navigation and a component-based architecture without any framework dependency.

9.2 Key Components

- router.js - Hash-based SPA router (#login, #dashboard, #datasets, #charts, etc.)
- api.js - HTTP client with JWT cookie and bearer token support
- app.js - Application entry point and view orchestration
- components/ - Reusable UI: chart.js, modal.js, navbar.js, sidebar.js
- views/ - Page views: login, dashboard, datasets, chart-builder, templates, users, settings, audit, review

9.3 Design System

- CSS custom properties (variables) for theming
- Orange primary (#f97316) / Indigo secondary (#4f46e5) palette
- Light and dark themes with no FOUC (applied before render)
- Responsive: sidebar collapses on mobile, topbar appears
- Design tokens: consistent spacing, typography, shadows, border-radius

9.4 Chart Rendering

Charts are rendered using Plotly.js v2.35.0 loaded from CDN. The backend generates chart HTML via Plotly Python, and the frontend renders it in a container div. The SPA also supports client-side re-rendering for theme changes.

10. Testing

Chartly includes 25 automated tests across 4 test files, plus a production verification script. Tests use pytest with FastAPI's TestClient and temporary SQLite databases.

10.1 Test Coverage

Test File	Tests	Coverage Area
test_regression.py	8	RBAC, upload, rate limiting, security headers, CSRF
test_jwt_revocation.py	5	Token revocation, bearer/cookie auth, login cycle
test_hashing_migration.py	6	Argon2id hashing, SHA-256 migration, demo accounts
test_security_config.py	4	API docs, HSTS, cookie flags, production checks

10.2 Test Infrastructure

- conftest.py - Fixtures: temp DB, TestClient, login/auth helpers, upload cleanup
- helpers.py - Legacy SHA-256 hash generation for migration tests
- prod_checks.py - Production mode verification (docs, HSTS, cookie flags)

10.3 Running Tests

```
cd backend && python -m pytest tests -v
```

10.4 End-to-End Smoke Test

A manual smoke test validates the full workflow: login -> upload CSV -> preview -> generate chart -> render chart -> delete. This test runs automatically in CI and after each deployment.

11. Deployment

11.1 Production Environment

- Platform: Vercel (free Hobby tier)
- Database: Neon Postgres (free tier, 0.5 GB storage)
- URL: https://chartly-sigma.vercel.app
- Auto-deploy: Push to main branch triggers Vercel build

11.2 Deployment Architecture

Vercel detects backend/main.py as the FastAPI entrypoint and bundles it as a serverless function. Static assets (HTML, CSS, JS) are served from Vercel's CDN via the public/ directory. The function handles all /api/* routes while CDN serves /static/* files.

11.3 Environment Variables

Variable	Value
DATABASE_URL	PostgreSQL connection string (Neon)
SECRET_KEY	JWT signing secret
ENVIRONMENT	production
COOKIE_SECURE	true (HTTPS)
TRUST_PROXY	true (Vercel behind proxy)
ALLOWED_ORIGINS	https://chartly-sigma.vercel.app

11.4 Deployment Steps

- 1. Create Neon Postgres database and obtain connection string
- 2. Push code to GitHub repository
- 3. Import project in Vercel dashboard (auto-detects FastAPI)
- 4. Configure environment variables in Vercel settings
- 5. Deploy - Vercel auto-builds and routes traffic
- 6. Verify: GET / returns HTML, POST /api/auth/login works

11.5 Deployment History

Commit	Description
69383a2	Vercel+Neon deployment wiring
8f3f556	Removed invalid runtime config
ac7e25a	Added includeFiles (did not resolve issue)
3972aca	Copied static to public/ for CDN

2f232e6

Set framework: fastapi explicitly

3d5c4f2

Serve index.html from bundled path

e99a563

Inline HTML, add public/index.html

12. Known Limitations

- No real-time collaboration (single-user per session)
- No WebSocket support for live data updates
- SQLite used locally does not support concurrent writes
- Chart rendering is server-side (Plotly Python) - adds latency vs client-side only
- No data transformation/cleaning pipeline (raw data only)
- No scheduled reports or email notifications
- No API key authentication for external integrations
- Free tier Neon database has 0.5 GB storage limit
- Vercel serverless functions have 10s timeout on free tier

13. Future Improvements

13.1 Short-term

- Data transformation pipeline (filter, join, pivot)
- Chart sharing via public links
- Dashboard creation (multiple charts per view)
- Export to PNG/PDF
- CSV/Excel export of chart data

13.2 Medium-term

- Real-time collaboration with WebSockets
- Custom SQL query editor
- Chart scheduling (daily/weekly email reports)
- Team workspaces with shared datasets
- API key management for external integrations

13.3 Long-term

- Machine learning integration (auto-insights, anomaly detection)
- Natural language chart queries (text-to-chart)
- Embed support (iframe snippets for dashboards)
- Multi-tenant architecture
- Plugin system for custom chart types